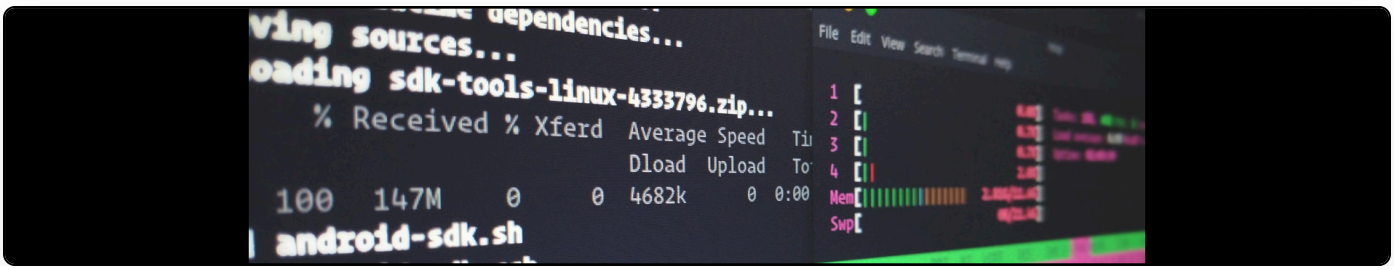
Continuing javascript execution after total DOM body replacement using DOM mutation observers

Cite as: Martin Paul Eve, "Continuing javascript execution after total DOM body replacement using DOM mutation observers", *eve.gd: Martin Paul Eve*, July 12, 2016, https://doi.org/10.59348/s2e01-4fy43.



As part of the translation platform we're building, I needed to implement the following workflow:

- If the DOM has been modified previously, then restore the DOM and run the substitution function.
- If the DOM hasn't been modified, then just run the substitution function.

The problem was that whenever I ran $("body").html(this.original_document); in the first of these cases, the javascript would stop executing. I think this is because the object that triggered the event was destroyed, even though the script was in the head tag.

In other words, whenever I replace the body tag of the HTML, my javascript immediately stops and I can't call the next function. I also tried using setTimeout to somehow defer this into a space that wouldn't be picked up by garbage collection or anything else.

I eventually worked out that I can solve this problem by using DOM Mutation Observers. The first thing I do, now, when the page loads is setup a DOM Mutation Observer thus (I'm using a mixture of coffeescript and javascript here):

```
this.continue_action = ""
`
var target = document.body;
var observer = new MutationObserver(this.handle_state_continuity);
var config = { attributes: true, childList: true, characterData: true };

observer.observe(target, config);
`
```

This basically says call the handle_state_continuity function whenever the contents of the body changes at any depth, in any way. Inside the event-firing method, I then have:

```
  startSubstitution: (event = {}) =>
    if this.original_document == ""
      this.original_document = `$("body").html()`

    if this.original_document != `$("body").html()`
      this.continue_action = "substitute"
      console.log("Annotran: Restoring DOM to original state.")
      `
      $("body").html(this.original_document);
      `
      return null
    else
      console.log("Annotran: Starting substitution directly.")
      this.makeSubstitution(event)
```

The important part to note here is that, in the case where it changes the body html, it first preserves the next action: this.continue_action = “substitute”.

The final part needed to glue this all together is the handle_state_continuity function, which for me looks like this:

```
  handle_state_continuity: (mutation = null) =>
    if this.continue_action != ""
      this.continue_action = ""
      console.log("Annotran: Starting substitution via state machine.")
      this.makeSubstitution(null)
```

And, tada! Now, when startSubstitution is called twice, the console output is:

Annotran: Starting substitution directly. [first time] Annotran: Restoring DOM to original state. [second time] Annotran: Starting substitution via state machine.